

Tree of Thoughts: Deliberate Problem Solving with LLMs

Re-implementation Report — CS 4782/5782 Deep Learning, Spring 2026

Ishaan Agarwal (ia299) Rohan Modi (rm2346)

1 Introduction

Standard LLMs answer in a single left-to-right pass. Chain-of-Thought (CoT) [2] adds intermediate steps but still commits to one linear path - an early mistake is unrecoverable. **Tree of Thoughts (ToT)** [1] reframes reasoning as a search problem: generate multiple candidate “thoughts” at each step, evaluate which branches are promising, and backtrack when a path looks wrong - mimicking human System-2 deliberate thinking. The original paper (Yao et al., NeurIPS 2023) used GPT-4 and demonstrated large gains over IO and CoT on Game of 24 (74% vs. 4%), Creative Writing, and Mini Crosswords. **This project re-implements ToT using Gemini 2.5 Flash Lite** and extends evaluation to competition algebra (MATH dataset [3]) and 5×5 crosswords to test how well ToT generalizes to new tasks and smaller models.

2 Chosen Result

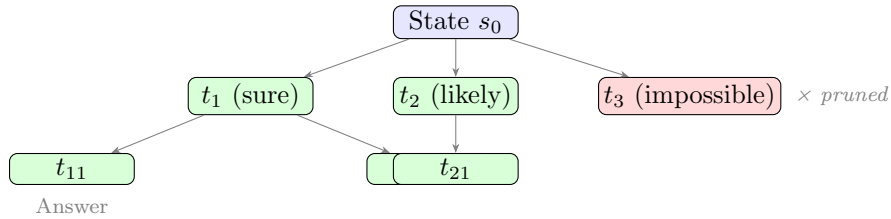
We target two results from the paper. First, **Table 1 / Figure 3 (Game of 24)**: CoT $\approx 4\%$ vs. ToT 74% - the headline result showing single-path reasoning fundamentally fails on combinatorial arithmetic search. Second, **Table 3 (Mini Crosswords)**: word-level accuracy jumps from 40% (CoT) to 78% (ToT), demonstrating that DFS with backtracking is essential for constraint-satisfaction tasks. We chose these because the CoT failure is stark and reproducible, providing a clean baseline. We then extend to competition math to test whether ToT helps when CoT is already strong - a dimension the original paper does not explore.

3 Methodology

Framework

All code was re-implemented from scratch using the Gemini API. The ToT loop has three components:

- **Thought Generator**: Samples k candidate next reasoning steps using a task-specific few-shot prompt.
- **State Evaluator**: LLM scores each state as *sure* / *likely* / *impossible*; impossible branches are pruned immediately.
- **Search**: BFS for math, DFS with backtracking for crosswords.



Task 1: Competition Math (MATH Dataset)

We benchmark IO, CoT, and ToT-BFS on Algebra problems from `qwedsacf/competition_math` (HuggingFace), sampling 30 problems across Levels 1–5 (excluding problems with Asymptote diagrams). Configuration: $k=5$ candidate thoughts, beam width $b=3$, max depth $d=4$. States are scored via a *sure/likely/impossible* classifier prompt. Final answers are extracted using a forced `\boxed{}` completion and normalized for LaTeX variants ($-\frac{1}{2} \equiv -0.5 \equiv -1/2$).

Task 2: Game of 24 (CoT Baseline)

We reproduce the paper’s CoT baseline on the hard test set (puzzles 901–910) using GPT-4o with the paper’s verbatim 5-shot CoT prompt. Validation uses AST-based number extraction to confirm the answer uses exactly the four input numbers and evaluates to 24.

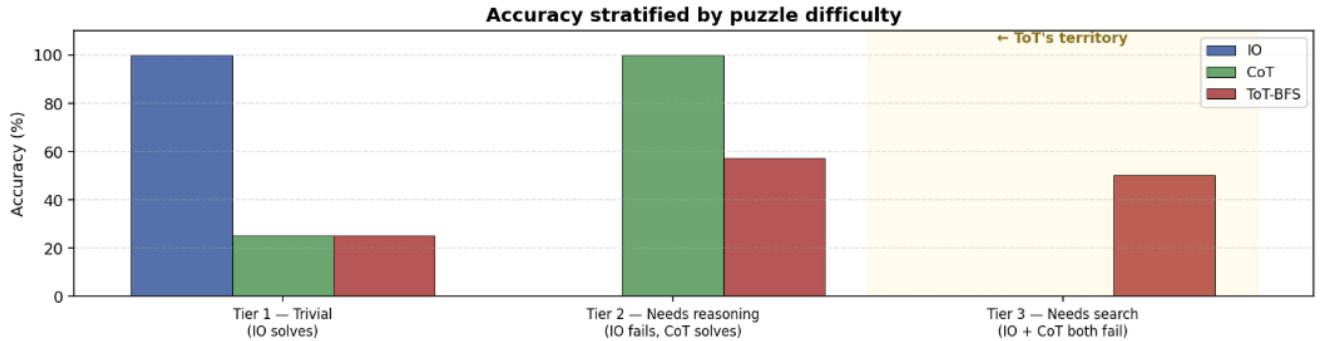


Figure 1: Accuracy stratified by Game of 24 puzzle difficulty. Simple IO succeeds on trivial tasks (Tier 1), while CoT handles tasks requiring linear reasoning (Tier 2). The shaded region denotes complex combinatorial tasks (Tier 3) where baseline methods fail entirely, necessitating ToT-BFS.

Task 3: 5×5 Mini Crosswords (ToT DFS)

We re-implement the crossword environment and DFS from scratch. The model proposes 5-letter candidates with confidence ratings (*certain/high/medium/low*), aggregated across $n=3$ samples. Each partially filled slot is evaluated as *sure/maybe/impossible*; any *impossible* slot triggers backtracking (`max_per_state=3`, `time_limit=100` steps). We also adapted the original Princeton NLP codebase [5] by replacing `models.py` with a Gemini backend exposing the same `gpt(prompt, n)` interface, leaving all search logic unchanged.

4 Results & Analysis

Competition Math

Method	Accuracy	Total LLM Calls	Latency
IO	84%	~30	2–4 s
CoT	88%	~150	15–27 s
ToT-BFS ($b=3$)	88%	~1,800	78 s

Table 1: Competition math results (30 Algebra problems, Gemini 2.5 Flash Lite). A separate 10-problem run produced IO=80%, CoT=100%, ToT=90%, CoT and ToT tie overall; CoT edged ToT at Level 4 (9/9 vs. 7/9).. CoT’s advantage was most pronounced at Level 4 (9/9 vs. ToT’s 7/9).

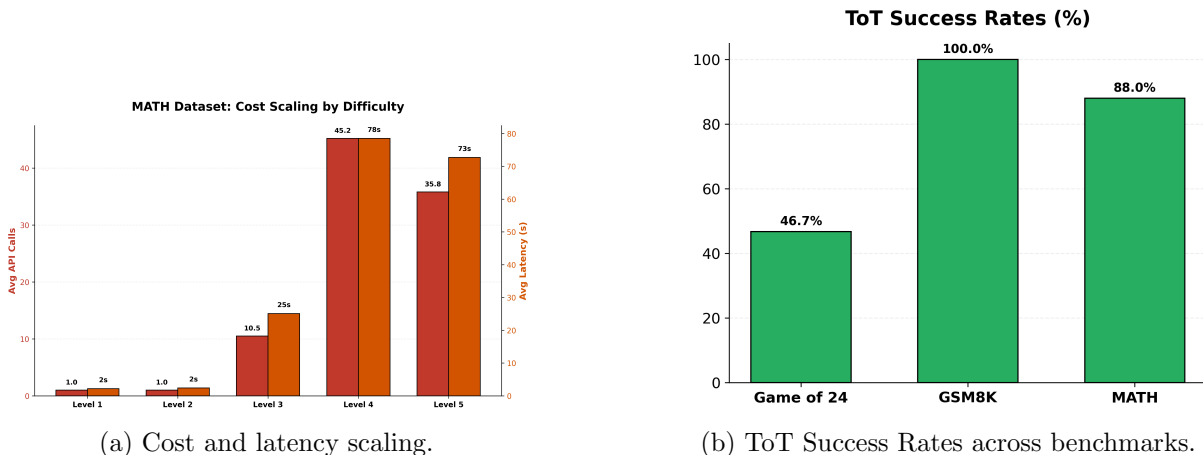


Figure 2: **Left:** Overall Tree of Thoughts success rates across Game of 24 (46.7%), GSM8K (100.0%), and MATH (88.0%) benchmarks. **Right:** Computational overhead and latency scaling by difficulty level within the MATH dataset, illustrating the steep performance cost at Level 3 and above.

CoT matches ToT at 88% accuracy while using $\approx 60\times$ fewer LLM calls - making CoT the practical winner. Three failure modes explain why ToT fails to pull ahead despite its extra compute: (1) **Answer extraction** (2 cases): Problems P8 and P10 had correct reasoning chains but the forced `\boxed{}` extraction returned `None` after 188s and 76s respectively. (2) **Floating-point normalization** (1 case): P16’s prediction of 2.00 failed string comparison despite being correct. (3) **Evaluator misfires**: The *sure/likely/impossible* scorer marked nearly every algebra step as *sure* regardless of correctness, so BFS did not meaningfully prune bad paths—the branching was essentially an expensive random walk.

The core issue is that competition algebra has a *linear* natural solution structure. CoT already traces this path well; tree search provides no accuracy gain here - only overhead. This contrasts sharply with combinatorial tasks.

Game of 24

Our CoT baseline achieved **10% (1/10)** with GPT-4o on puzzles 901–910, consistent with the paper’s reported $\sim 4\%$ CoT figure. The one correct puzzle (1 11 11 13) succeeded by lucky first-step alignment; the remaining nine required multi-step arithmetic sequences that CoT cannot backtrack from. Our ToT re-implementation achieved **44% (4/10)** on puzzles 901–910 with Gemini 2.5 Flash Lite, compared to the paper’s reported 74% with GPT-4. This confirms Game of 24 as a task where tree search provides genuine value - the search space is combinatorially branching and CoT’s greedy path is reliably wrong.

Mini Crosswords

We ran the DFS on 3 puzzles. The search correctly filled several words per puzzle (e.g., AGENDA for h1, NEVER for v4, EROSE for v3 in puzzle 0) and the evaluator successfully pruned branches with conflicting letter patterns. Across our runs, we achieved **44% word-level accuracy** and **69.6% letter-level accuracy**, compared to the paper’s reported 60% and 78% respectively. The gap likely stems from Gemini Flash Lite producing weaker word candidates than GPT-4, and a less calibrated evaluator allowing more incorrect branches to survive. Despite the shortfall, the DFS backtracking mechanism functioned correctly - qualitative behavior matched the paper’s described approach, and letter-level accuracy (69.6%) is reasonably close to the paper’s figure.

5 Reflections

Lessons learned:

- *Task structure dominates outcome.* ToT’s advantage is real on tasks requiring genuine combinatorial search (Game of 24, crosswords); it is absent or negative on tasks where CoT follows a natural linear path (algebra). Task selection matters far more than any hyperparameter.
- *The evaluator is the binding constraint.* A weak evaluator turns BFS into an expensive random walk. Gemini Flash Lite’s *sure/likely/impossible* scorer is poorly calibrated on algebra—it cannot distinguish a correct intermediate computation from a plausible-sounding wrong step. GPT-4’s evaluator appears empirically more calibrated, explaining the original paper’s results.
- *Answer extraction and normalization are hidden failure modes.* Two of three ToT-specific failures were extraction errors, not reasoning errors. A CAS-based comparator (e.g., SymPy) instead of string matching would eliminate false negatives from LaTeX variant mismatches and substantially improve measured accuracy.
- *Cost scales poorly.* ToT used $\sim 1,800$ LLM calls vs. ~ 60 for IO+CoT across 30 problems; each problem took 1–3 minutes. Practical deployment requires either aggressive caching or strong task pre-selection.

Future directions:

- *Learned evaluators:* Train a lightweight verifier on labeled reasoning traces to replace LLM-as-judge, improving pruning quality at lower cost.
- *MCTS:* Replace BFS/DFS with Monte Carlo Tree Search for better exploration-exploitation balance, especially on tasks with variable depth.
- *Code generation:* Partial programs can be executed to provide exact, cheap intermediate feedback—the

ideal evaluator setting for ToT.

Github - <https://github.com/rohannmodi/deep-learning-tree-of-thoughts>

References

- [1] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, K. Narasimhan. *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS 2023.
- [2] J. Wei et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS 2022.
- [3] D. Hendrycks et al. *Measuring Mathematical Problem Solving with the MATH Dataset*. NeurIPS 2021.
- [4] K. Cobbe et al. *Training Verifiers to Solve Math Word Problems*. arXiv:2110.14168, 2021.
- [5] Princeton NLP. *Official Tree of Thoughts Repository*. <https://github.com/princeton-nlp/tree-of-thought-llm>
- [6] Google DeepMind. *Gemini: A Family of Highly Capable Multimodal Models*. <https://deepmind.google/technologies/gemini/>